

Warsztat: Wektory w PostgreSQL (pgvector)

1. Wstęp

```
-- Sprawdź dostępność rozszerzenia
SELECT * FROM pg_available_extensions WHERE name = 'vector';

-- Włącz rozszerzenie w bieżącej bazie
CREATE EXTENSION IF NOT EXISTS vector;
```

2. Podstawy pracy z wektorami

```
CREATE TABLE dokumenty (
    id SERIAL PRIMARY KEY,
    tytuł TEXT NOT NULL,
    tresc TEXT,
    embedding VECTOR(3)
);

INSERT INTO dokumenty (tytuł, tresc, embedding) VALUES
('Kot', 'Zwierzę domowe', '[0.1, 0.2, 0.3]'),
('Pies', 'Zwierzę domowe', '[0.15, 0.25, 0.35]'),
('Samochód', 'Pojazd', '[0.9, 0.8, 0.7]');
```

Wyszukiwanie najbliższych wektorów względem zadanego wektora:

Dodaj kod, który wyszuka najbliższe wektory względem wektora: [0.1, 0.2, 0.3]

Najważniejsze operatory pgvector:

- <-> — odległość euklidesowa (L2).
- <#> — ujemny iloczyn skalarny.
- <=> — odległość kosinusowa.

3. Funkcje i procedury

Funkcja wyszukująca podobne dokumenty:

Stwórz funkcję dwuargumentową (wektor, limit wyników), który wyszuka podobne wektory.

Wywołanie funkcji:

```
SELECT * FROM znajdz_podobne('[0.1, 0.2, 0.3]', 2);
```

Procedura dodająca dokument i wpis do logu:

```
CREATE TABLE logi (  
  id SERIAL PRIMARY KEY,  
  opis TEXT,  
  kiedy TIMESTAMP DEFAULT now()  
);
```

Stwórz procedurę, która pozwoli na dodanie nowego dokumentu wraz z dodaniem informacji o tym fakcie w tabeli logów.

Wywołanie procedury:

```
CALL dodaj_dokument('Rower', 'Pojazd jednośladowy', '[0.85, 0.75, 0.65]');
```

4. Wyzwalacze (triggery)

```
CREATE TABLE historia_zmian (  
  id SERIAL PRIMARY KEY,  
  dokument_id INT,  
  stary_embedding VECTOR(3),  
  nowy_embedding VECTOR(3),  
  zmieniono TIMESTAMP DEFAULT now()  
);
```

Stwórz wyzwalacz, który będzie kontrolował pełną historię zmian w tabeli dokumenty

Stwórz drugi wyzwalacz, który nie pozwoli na dodanie wektora zerowego: [0,0,0]